

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-211БВ-24

Студент: Антыгин В.Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 5.11.25

Москва, 2025

Постановка задачи

Вариант 19

Необходимо создать динамические библиотеки, реализующие следующие функции:

- Функция 1: Подсчёт количества простых чисел на отрезке $[a, b]$. Сигнатура: `int prime_count(int a, int b)`
 - Реализация №1: Наивный алгоритм. Проверить делимость текущего числа на все предыдущие числа.
 - Реализация №2: Решето Эратосфена.
- Функция 2: Подсчёт площади плоской геометрической фигуры по двум сторонам. Сигнатура: `float area(float a, float b)`
 - Реализация №1: Фигура прямоугольник ($S = a * b$)
 - Реализация №2: Фигура прямоугольный треугольник ($S = 0.5 * a * b$)

Требуется использовать библиотеки двумя способами:

- Программа №1: Статическая линковка на этапе компиляции
- Программа №2: Динамическая загрузка в runtime с возможностью переключения между реализациями

Пользовательский ввод организован следующим образом:

- Команда 0 – переключить реализацию (только для программы №2)
- Команда 1 `arg1 arg2` – вызов первой функции `prime_count` с аргументами
- Команда 2 `arg1 arg2` – вызов второй функции `area` с аргументами

Общий метод и алгоритм решения

Вариант 19

Необходимо создать динамические библиотеки, реализующие следующие функции:

Функция 1: Подсчёт количества простых чисел на отрезке $[a, b]$ (сигнатура: `int prime_count(int a, int b)`)

- Реализация №1: Наивный алгоритм. Проверить делимость текущего числа на все предыдущие числа.
- Реализация №2: Решето Эратосфена.

Функция 2: Подсчёт площади плоской геометрической фигуры по двум сторонам (сигнатура: `float area(float a, float b)`)

- Реализация №1: Фигура прямоугольник ($S = a \times b$)
- Реализация №2: Фигура прямоугольный треугольник ($S = 0.5 \times a \times b$)

Требуется использовать библиотеки двумя способами:

- Программа №1: Статическая линковка на этапе компиляции
- Программа №2: Динамическая загрузка во время исполнения с возможностью переключения между реализациями

Пользовательский ввод организован следующим образом:

- Команда 0 – переключить реализацию (только для программы №2)
- Команда 1 `arg1 arg2` – вызов первой функции `prime_count` с аргументами
- Команда 2 `arg1 arg2` – вызов второй функции `area` с аргументами

Код программы

contract.h

```
#ifndef CONTRACT_H
#define CONTRACT_H

// контракт на подсчёт простых чисел
typedef int (*prime_count_func)(int a, int b);

// контракт на площадь
typedef float (*area_func)(float a, float b);

#endif
```

lib_naive.c

```
#include "contract.h"

int prime_count(int a, int b) {
    if (a > b) {
        int t = a;
        a = b;
        b = t;
    }

    int count = 0;

    for (int x = a; x <= b; x++) {
        if (x < 2)
            continue;

        int is_prime = 1;
        for (int d = 2; d < x; d++) {
            if (x % d == 0) {
                is_prime = 0;
                break;
            }
        }

        count += is_prime;
    }

    return count;
}

float area(float a, float b) {
    return a * b;
}
```

lib_opt.c

```
#include "contract.h"
```

```
#include <stdlib.h>
```

```
int prime_count(int a, int b) {  
    if (a > b) {  
        int t = a;  
        a = b;  
        b = t;  
    }  
  
    if (b < 2)  
        return 0;  
  
    int n = b;  
    char* is_prime = calloc(n + 1, 1);  
  
    for (int i = 2; i <= n; i++)  
        is_prime[i] = 1;  
  
    for (int i = 2; i * i <= n; i++) {  
        if (!is_prime[i])  
            continue;  
        for (int j = i * i; j <= n; j += i)  
            is_prime[j] = 0;  
    }  
  
    int count = 0;  
    for (int x = a; x <= b; x++)  
        if (x >= 2 && is_prime[x])  
            count++;  
  
    free(is_prime);  
    return count;  
}
```

```
float area(float a, float b) {  
    return 0.5f * a * b;  
}
```

prog1.c

```
#include "contract.h"
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Функции из библиотеки в компайл тайм (например libnaive.so)
```

```
extern int prime_count(int a, int b);
extern float area(float a, float b);
```

```
int main() {
    printf("Программа 1: Статическая линковка с библиотекой\n");
    printf("Команды:\n");
    printf(" 1 <a> <b> - подсчет простых чисел на отрезке [a, b]\n");
    printf(" 2 <a> <b> - вычисление площади (a, b - стороны)\n");
    printf(" 0      - выход\n\n");

    char input[512];
    int cmd, a, b;
    float fa, fb;

    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin))
            break;

        input[strcspn(input, "\n")] = 0;

        if (sscanf(input, "%d", &cmd) != 1) {
            printf("Неверная команда\n");
            continue;
        }

        if (cmd == 0) {
            printf("Выход.\n");
            break;
        } else if (cmd == 1) {
            if (sscanf(input, "%d %d %d", &cmd, &a, &b) != 3) {
                printf("Формат: 1 <a> <b>\n");
                continue;
            }
            int result = prime_count(a, b);
            printf("Количество простых чисел на [%d, %d]: %d\n", a, b, result);
        } else if (cmd == 2) {
            if (sscanf(input, "%d %f %f", &cmd, &fa, &fb) != 3 || fa <= 0 || fb <= 0) {
                printf("Формат: 2 <a> <b> (a, b > 0)\n");
                continue;
            }
            float result = area(fa, fb);
            printf("Площадь: %.4f\n", result);
        } else {
            printf("Неизвестная команда\n");
        }
    }

    return 0;
}
```

prog2.c

```
#include "contract.h"
```

```
#include <ctype.h>
```

```
#include <dlfcn.h>
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
static prime_count_func current_prime_count = NULL;
```

```
static area_func current_area = NULL;
```

```
static void* current_lib = NULL;
```

```
// Текущая загруженная библиотека (0 = naive, 1 = opt)
```

```
static int current_impl = 0;
```

```
static const char* lib_paths[] = {".libnaive.so", ".libopt.so"};
```

```
static int prime_stub(int a, int b) {
```

```
    (void)a;
```

```
    (void)b;
```

```
    fprintf(stderr, "Ошибка: функция prime_count не загружена!\n");
```

```
    return 0;
```

```
}
```

```
static float area_stub(float a, float b) {
```

```
    (void)a;
```

```
    (void)b;
```

```
    fprintf(stderr, "Ошибка: функция area не загружена!\n");
```

```
    return 0.0f;
```

```
}
```

```
int load_lib(const char* path) {
```

```
    if (current_lib) {
```

```
        dlclose(current_lib);
```

```
        current_lib = NULL;
```

```
    }
```

```
    current_lib = dlopen(path, RTLD_LAZY);
```

```
    if (!current_lib) {
```

```
        fprintf(stderr, "Не удалось загрузить %s: %s\n", path, dlerror());
```

```
        current_prime_count = prime_stub;
```

```
        current_area = area_stub;
```

```
        return 0;
```

```
    }
```

```
// Сброс ошибок
```

```
dlerror();
```

```
// Загрузка prime_count
```

```

current_prime_count = (prime_count_func)dlsym(current_lib, "prime_count");
char* err = dlerror();
if (err) {
    fprintf(stderr, "Символ 'prime_count' не найден: %s\n", err);
    current_prime_count = prime_stub;
}

// Загрузка area
current_area = (area_func)dlsym(current_lib, "area");
err = dlerror();
if (err) {
    fprintf(stderr, "Символ 'area' не найден: %s\n", err);
    current_area = area_stub;
}

printf("Успешно загружена библиотека: %s\n", path);
return 1;
}

void switch_implementation() {
    current_impl = 1 - current_impl; // Переключение 0/1
    printf("Переключение на реализацию %d (%s)\n", current_impl + 1,
        current_impl == 0 ? "наивная" : "оптимизированная");
    load_lib(lib_paths[current_impl]);
}

int main(int argc, char** argv) {
    printf("Программа 2: Динамическая загрузка (runtime)\n");
    printf("Команды:\n");
    printf(" 0      - переключить реализацию\n");
    printf(" 1 <a> <b> - подсчет простых чисел на отрезке [a, b]\n");
    printf(" 2 <a> <b> - вычисление площади (a, b - стороны)\n\n");

    current_prime_count = prime_stub;
    current_area = area_stub;

    // Первая библиотека по умолчанию
    if (argc > 1) {
        load_lib(argv[1]);
    } else {
        load_lib(lib_paths[0]);
    }

    char input[512];
    int cmd, ia, ib;
    float fa, fb;

    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin))

```



```

    break;

input[strcspn(input, "\n")] = 0;

if (sscanf(input, "%d", &cmd) != 1) {
    printf("Неверная команда\n");
    continue;
}

if (cmd == 0) {
    // Переключение реализации
    switch_implementation();
} else if (cmd == 1) {
    // Подсчет простых чисел
    if (sscanf(input, "%d %d %d", &cmd, &ia, &ib) != 3) {
        printf("Формат: 1 <a> <b>\n");
        continue;
    }
    int result = current_prime_count(ia, ib);
    printf("Количество простых чисел на [%d, %d]: %d\n", ia, ib, result);
} else if (cmd == 2) {
    // Вычисление площади
    if (sscanf(input, "%d %f %f", &cmd, &fa, &fb) != 3 || fa <= 0 || fb <= 0) {
        printf("Формат: 2 <a> <b> (a, b > 0)\n");
        continue;
    }
    float result = current_area(fa, fb);
    printf("Площадь: %.4f\n", result);
} else {
    printf("Неизвестная команда\n");
}
}

if (current_lib)
    dlclose(current_lib);

return 0;
}

```

Протокол работы программы

```
OS/lab4 → make
gcc -Wall -Wextra -Werror -fPIC -shared -o libnaive.so src/lib_naive.c -lm
gcc -Wall -Wextra -Werror -fPIC -shared -o libopt.so src/lib_opt.c -lm
gcc -Wall -Wextra -Werror -fPIC -o prog1 src/prog1.c -L. -lnaive -lm -Wl,-rpath,.
gcc -Wall -Wextra -Werror -fPIC -o prog2 src/prog2.c -ldl -lm
OS/lab4 → ./prog1
Программа 1: Статическая линковка с библиотекой
Команды:
  1 <a> <b> - подсчет простых чисел на отрезке [a, b]
  2 <a> <b> - вычисление площади (a, b - стороны)
  0        - выход

> 1 69 420
Количество простых чисел на [69, 420]: 62
> 2
Формат: 2 <a> <b> (a, b > 0)
> 2 11 11
Площадь: 121.0000
> 0
Выход.
OS/lab4 → ./prog2
Программа 2: Динамическая загрузка (runtime)
Команды:
  0 - переключить реализацию
  1 <a> <b> - подсчет простых чисел на отрезке [a, b]
  2 <a> <b> - вычисление площади (a, b - стороны)

Успешно загружена библиотека: ./libnaive.so
> 2 7 6
Площадь: 42.0000
> 0
Переключение на реализацию 2 (оптимизированная)
Успешно загружена библиотека: ./libopt.so
> 2 7 6
Площадь: 21.0000
> 1 1 10000000
Количество простых чисел на [1, 10000000]: 664579
```

Вывод

Выполнив эту работу, я приобрел навыки работы с динамическими библиотеками в Linux. Освоил использование функций `dlopen`, `dlsym`, `dlclose` и `dlerror` для загрузки библиотек в runtime. Понял разницу между флагами компиляции `-fPIC` и `-shared`, а также научился использовать флаги линковки `-L`, `-l` и `-Wl,-rpath` для корректной работы с динамическими библиотеками.

Основное отличие статической линковки от динамической загрузки заключается в моменте разрешения символов. При статической линковке символы разрешаются на этапе компиляции, и библиотека загружается автоматически при старте программы. При динамической загрузке программист явно контролирует загрузку библиотеки в процессе выполнения программы через API операционной системы.

Динамическая загрузка обеспечивает большую гибкость: возможность переключения между реализациями без перекомпиляции программы, что полезно для создания модульных и расширяемых систем. Однако это требует дополнительной обработки ошибок и использования указателей на функции.

Столкнулся с необходимостью тщательной обработки ошибок при загрузке библиотек. Использование функций-заглушек предотвращает ошибки сегментации при попытке вызова невалидных указателей на функции, что повышает надежность программы.